Computer Security Symposium 2026
19-23 October 2026

Attestable Audit: コード非開示下でのプロパティベースアテステーション

今村 拓万[1,a)]

概要：Trusted Execution Environment（TEE）はリモートアテステーションを通じて自己の完全性を証明できる．TEE は，内部で動作するコードの測定値を含む構成情報にハードウェアに紐付く鍵で署名し，アテステーションレポートを発行する．検証者はレポート内の測定値をリファレンス値と照合することで整合性を検証する．しかし，検証者がハッシュ化前データにアクセスできない場合，特定の測定値を持つ何かが TEE 内で実行されていることは確証できても，プログラムが期待通りに動作しているかは確証できない．そのため，プロプライエタリな環境では，リファレンス値提供者を完全に信頼せざるを得ない．本研究は，2 つの TEE アプリケーション，すなわち *Attestable Build*（Hugenroth *et al.*, 2025）と *Attestable Audit*（本論文で提案）を組み合わせ，ハッシュ化前データを開示することなくこのギャップを埋めるアーキテクチャを提案する．これらの TEE が発行するレポートにより，**コードを開示することなく**，監査対象コードの特性を証明できる．

キーワード：プロパティベースアテステーション，TEE，ゼロ知識証明，コード監査，アテスタブルビルド

Attestable Audit: Property-based attestation without code disclosure

Takuma Imamura[1,a)]

Abstract:
Trusted Execution Environments (TEEs) can prove their own integrity through remote attestation: a TEE issues an attestation report by signing, with a hardware-bound key, its configuration information including measurements of the code running inside it. A verifier validates the integrity by comparing the measurements in the report against reference values. However, when the verifier lacks access to the pre-hashed data, it can only confirm that something with a given measurement is running inside the TEE—not that the program behaves as expected—forcing full trust in the reference value provider in proprietary settings. We propose an architecture that closes this gap without disclosing the pre-hashed data, combining two TEE applications: *Attestable Build* (Hugenroth *et al.*, 2025) and *Attestable Audit* (proposed in this paper). The attestation reports issued by these TEEs make it possible to attest properties of the audited code *without code disclosure*.

Keywords: Property-Based Attestation, TEE, Zero-Knowledge Proof, Code Audit, Attestable Build

1. はじめに

1.1 背景

Trusted Execution Environment（TEE）におけるアテステーション（Attestation）は，TEE のコード及びデータの整合性及び機密性をリモートピアに対して証明するプロトコルである．以下，本稿では RATS アーキテクチャ [3] の用語に従う．RATS は次の役割（role）を定める．*Attester* は，自身の状態に関する主張を含むエビデンス（Evidence）を発行する，アテステーション対象のプラットフォームである．*Verifier*（検証者）はエビデンスを評価し，その評価結果（Attestation Results）を *Relying Party*（依拠当事者；評価結果に基づき Attester と相互作用するか否かを判

[1] 株式会社 Acompany，Confidential Computing Lab
Confidential Computing Lab, Acompany Co., Ltd.
[a)] takuma.imamura@acompany-ac.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

断する当事者）に伝える．この評価に際して Verifier は，*Endorser*（典型的にはハードウェアベンダ）が発行する証明書チェーンや証明書失効リストなどからなるエンドースメント（Endorsements）を用いてエビデンスの真正性を検証し，*Reference Value Provider*（リファレンス値プロバイダ；典型的にはソフトウェアベンダ）が提供するリファレンス値（Reference Value）とエビデンス内の測定値とを照合する．

アテステーションの基本的な流れは次の通りである．Attester は，隔離実行コンテキストにロードされたコードの暗号学的ハッシュ値を測定し，この測定値（measurement）を含むシステムの構成情報に対して当該 TEE プラットフォームに紐付く署名鍵（Attestation Key）によって署名を施すことで，エビデンスとしてアテステーションレポート（Attestation Report）を発行する．Verifier は，エンドースメントを用いてレポートの署名検証等の真正性検証を行った後，レポート内測定値をリファレンス値と照合することで，期待されるコードが TEE 内で動作していることを確証する．

この測定→検証というアテステーションのパターンは，TEE エコシステム全体において複数の形態をとりながらも共通して現れる．

最も基本的な例は TEE 初期状態のアテステーションである．Intel SGX では，SGX エンクレーブイメージの初期メモリ配置のハッシュ値がロード時に測定され，レポート（SGX quote）の `MRENCLAVE` フィールドにバインドされる [8]．一方，仮想化ベースの TEE（Confidential VM, CVM）である AMD SEV-SNP や Intel TDX では，VM の初期メモリ状態のハッシュ値（Launch Measurement；それぞれ `MEASUREMENT` 及び `MRTD`）が VM 起動時に測定され，レポート（それぞれ SNP report 及び TD quote）にバインドされる．Verifier はこれら測定値をリファレンス値と照合し，期待される SGX エンクレーブ又は VM のイメージが動作していることを確認する．

前述のアテステーションは TEE 起動時の整合性を保証するのみである．起動後の整合性に関するアテステーションはより複雑な形をとる．CVM におけるゲスト OS のランタイム整合性は，通常，カーネルが提供するランタイム整合性システムによって保護される．Linux カーネルにおいては Integrity Measurement Architecture（IMA）[23] がこれを担う．IMA は，測定ポリシーに従って，アクセスされたファイルのパス及びハッシュ値を測定・記録する，カーネル整合性サブシステムである．IMA による測定ログ自体を検証可能にするため，IMA は各エントリをゲストとハイパーバイザの双方から隔離された追記専用レジスタへ extend する．そのようなレジスタとしては，ゲスト VM 内の特権レベル（例：SEV-SNP における VMPL 0 [16]）に配置された vTPM の Platform Configuration Register（PCR）や，Intel TDX が提供する Runtime Measurement Register（RTMR）[13] などが利用可能である．Verifier は，当該レジスタの値を用いて測定ログの真正性を検証した後，測定ログの各エントリを何らかの Appraise ポリシーに従って評価する．例えば，特定のファイルパス（ないしパスのパターン）とハッシュ（ないしハッシュのリスト）の組合せを許可又は拒否するルールの集合などが考えられる．拒否ポリシーは，深刻な脆弱性を含むことが知られているライブラリバージョンの排除や，既知のマルウェアのブロックに特に有用である．

Confidential Containers（CoCo）[7]，すなわち CVM 内で実行されるコンテナはさらに別の例を与える．先ずホストは，許可されたコンテナイメージのダイジェストなどを含むコンテナポリシーのハッシュダイジェストを，ゲスト VM 起動時に設定可能なレジスタ（SEV-SNP の `HOST_DATA` や TDX の `MR_CONFIG_ID` など）に設定し，対応するポリシーをゲスト内のポリシーエンジンに渡す．ポリシーエンジンは，受け取ったポリシーのハッシュをアテステーションレポート内のポリシー測定フィールド（例えば `HOST_DATA` や `MR_CONFIG_ID`）と比較することでその真正性を検証し，その後ポリシーに従ってコンテナを起動する．Verifier は以下の手順により検証を実施する．

(1) アテステーションレポート内の起動時測定値（例えば `MEASUREMENT` や `MRTD`）をリファレンス値と比較し，期待されるコンテナランタイムとポリシーエンジンが動作していることを確認する．
(2) アテステーションレポート内のポリシー測定フィールド（例えば `HOST_DATA` や `MR_CONFIG_ID`）を期待されるコンテナポリシーのハッシュと比較し，期待されるポリシーが CVM 内にロードされていることを確認する．
(3) ポリシーに埋め込まれたコンテナイメージのダイジェストをリファレンス値と比較し，期待されるコンテナが動作していることを確認する．

これらいずれの場合も，Attester は，VM 初期イメージ，ゲスト OS 内のカーネルモジュール・ライブラリ・実行ファイル，コンテナイメージなど，自身の動作定義のダイジェストを含むエビデンスを Verifier に提供し，Verifier はそのダイジェストをリファレンス値と比較することでエビデンスを検証する．

対象のプログラムがオープンソースである（厳密に言えば，そのビルドが再現可能である [14]）場合，Verifier はソースからビルドして得られるダイジェストを計算することにより，これらのリファレンス値を自ら用意できる．ソースが公開されていない場合であっても，実行ファイルやコンテナイメージ自体が公開されている限り，Verifier は独立にリファレンス値を導出できる．

一方，プログラムがクローズド／プロプライエタリな状

© 2026 Information Processing Society of Japan

況では，事情が根本的に異なる．この場合，Verifier は自力でリファレンス値を導出できない．代わりに，Reference Value Provider（通常はソフトウェアベンダ）がリファレンス値のみを Verifier に配布する．このとき Verifier は「このダイジェストを持つ何かが TEE 内で動作している」ことは確証できるが，それ以上のことは何も分からない．すなわち，実際にどのようなプログラムが動作しているのか，またそのプログラムが関心のある特性（例えばバックドアの不在）を持つかどうかは検証できない．これは Sadeghi and Stüble [22] で指摘された問題と本質的には同じものである．すなわち，バイナリのダイジェストは単なる識別子であって，意味論的保証ではない．よってプログラムの振る舞いに関しては Reference Value Provider を全面的に信頼せざるを得ない．

1.2 本研究の貢献

本研究は，ハッシュ化前データを開示することなく，アテステーションにおけるこのギャップを埋めるアーキテクチャを提案するものである．提案アーキテクチャは次の 2 つの TEE アプリケーションを組み合わせることによって実現される．

Attestable Build は，Hugenroth *et al.* [12] によって導入されたもので，TEE など[*1]のアテステーション可能な隔離環境内でソースコードをビルドすることにより，ソースコードのダイジェストをビルドアーティファクトのダイジェストに暗号学的にバインドする．

Attestable Audit は，TEE 内でソースコードに対する（半）自動監査を実行することにより，ソースコードのダイジェストをコードの特性に暗号学的にバインドする．ここでいう（半）自動監査には，ファジング，Valgrind [19]，テイント解析，AI コード監査，形式検証などが含まれる．

これら 2 つのメカニズムにより，動作中の TEE のアテステーションエビデンスにおいて Verifier が観測するビルドアーティファクトのダイジェストから，ソースコードの検証可能な特性へと至るチェーンが，ソースコードもビルドアーティファクトも開示することなく得られる．本アプローチの背景にあるアイデアは，TEE のアテステーションがハードウェアに裏付けられたゼロ知識証明（ZKP）と見做せるというものである（詳細な議論は [21] を参照）．

Attestable Audit の実現可能性を実証するため，Gramine [24], [25] を用いた Intel SGX 上の概念実証（PoC）コードを実装した．この PoC は Python スクリプトを入力として受け取り，静的セキュリティ解析ツールである Bandit [20] を SGX エンクレーブ内で実行する．その監査判定（verdicts）はソースコードのダイジェストとともに SGX quote にバインドされる．

[*1] 同論文は PoC として AWS Nitro Enclaves を採用している．Nitro Enclaves は TEE と同様の方法でアテステーション可能であるが，ハイパーバイザを信頼する点で TEE とは異なる．

2. システムモデル及び信頼仮定

本稿では標準的な RATS ロールモデル [3]（Section 1 参照）に従う．Relying Party は Verifier と Endorser を信頼する．Attester は RATS に準拠したアテスタブルなプラットフォームであれば何でもよく，その信頼の仮定は基盤となる技術から継承される．例えば，Attester としては，SGX エンクレーブ，CVM，TPM を持つ Linux システム，AWS Nitro Enclaves などを許容する．ソフトウェアベンダ（Attester 内で動作するアプリケーションの開発者）は Reference Value Provider として振る舞う．

標準モデルからの重要な相違点は，Relying Party が Reference Value Provider を信頼**しない**ことである．この信頼を除去するため，リファレンス値及び監査判定の生成を，Attestable Build を実行する *Attestable Builder* と，Attestable Audit を実行する *Attestable Auditor* という一対の TEE アプリケーションに委譲する．これらに対しては標準的な TEE 脅威モデル（ハードウェアの信頼の起点とチップベンダの PKI は信頼し，TEE 外の全ての特権ソフトウェアを信頼しない）を仮定する．

Reference Value Provider への信頼を完全に除去するためには，Attestable Builder 及び Auditor の挙動自体は独立に検証可能でなければならない．すなわち，それらのリファレンス値は公開の検査可能なもの，つまり公開されているビルドアーティファクト及び／又は公開されたソースコードから再現可能に導出されたものでなければならない．この要件は Attestable Builder 及び Auditor にのみ適用されることに注意する．**監査対象のアーティファクト自体は非開示のまま**である．

なお，Attestable Auditor がソースコードではなく実行ファイルを対象とする場合，Attestable Builder は不要であり，Attestable Auditor は実行ファイルに対して直接適用できる（例えば静的バイナリ解析）．

3. アーキテクチャ

本節では提案アーキテクチャを詳述する．図 1 にデータフローを示す．

以下の記法を用いる．$h_X = H(X)$ は，固定された暗号学的ハッシュ関数 H の下でのデータ X のハッシュダイジェストを表す．$m_X := \mathrm{meas}(X)$ は，アテステーションレポートにバインドされるアーティファクト X の測定値を表す（例えば，SGX エンクレーブでは `MRENCLAVE`，SEV-SNP や TDX の VM では起動時ダイジェスト，Linux IMA ではファイルハッシュダイジェスト）．$\mathrm{AR}_X(d)$ は，`REPORT_DATA` フィールドにデータ d がバインドされた，Attester X のアテステーションレポートを表す．

プロプライエタリなアーティファクトを S と表記する

© 2026 Information Processing Society of Japan

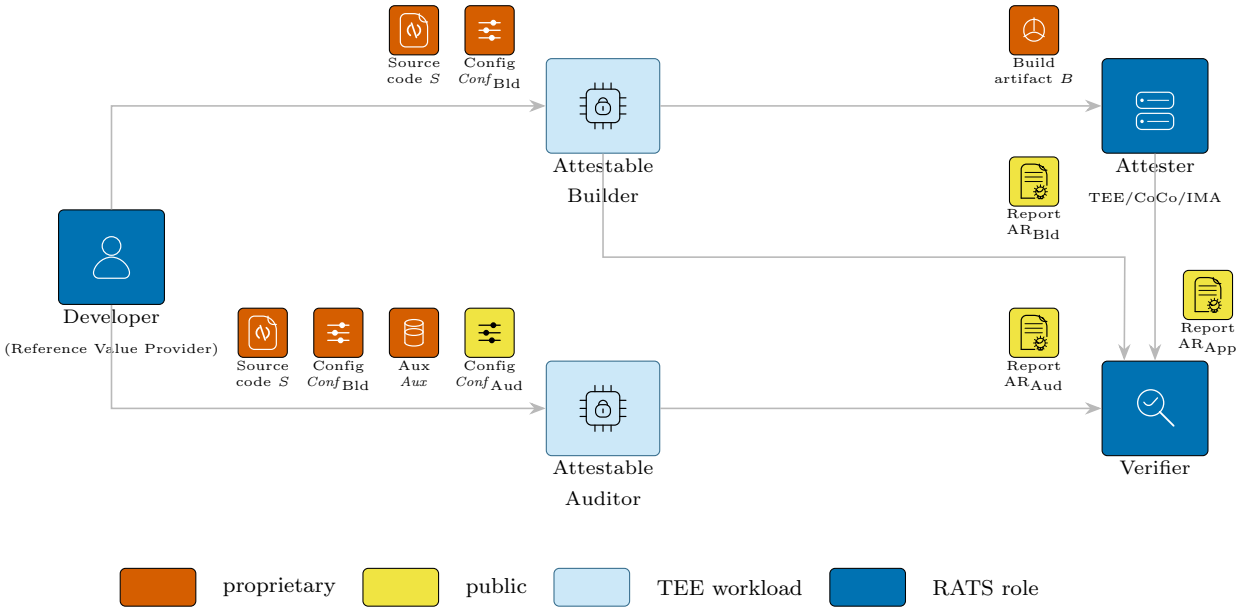


図 1　提案アーキテクチャのデータフロー.

(「ソースコード」と呼ぶが，本構成は内容に依存しない．S はソースコード，LLM の重みデータ，アプリケーションの設定，コンテナポリシーファイルなどでもよい)．ソースからビルドされないアーティファクトについては，Attestable Build を省略でき，Attestable Audit のみが必要となる．

3.1　Attestable Build

Hugenroth *et al.* [12] によって導入された Attestable Build を採用する．*Attestable Builder* は，ソースコードのダイジェストとビルドアーティファクトの測定値を結び付ける役割を担う．このコンポーネントはオープンソースかつ再現可能にビルドできるか，又はそのバイナリが公開されていなければならない．

Input　ソースコード S（非公開）；ビルド設定 $Conf_{\mathrm{Bld}}$（非公開）.

Processing　(1) $h_S := H(S)$ と $h_{Conf_{\mathrm{Bld}}} := H(Conf_{\mathrm{Bld}})$ を計算し；(2) ビルドを実行し；(3) 測定値 $m_B := \mathrm{meas}(B)$ を持つビルドアーティファクト B を得て；(4) 次のアテステーションレポートを発行する：

$$R_{\mathrm{Bld}} := \mathrm{AR}_{\mathrm{Bld}} \circ H(d_{\mathrm{Bld}}),$$
$$d_{\mathrm{Bld}} := \left(h_S, h_{Conf_{\mathrm{Bld}}}, m_B\right).$$

Output　未ハッシュのレポートデータ d_{Bld}（公開），アテステーションレポート R_{Bld}（公開），及びビルドアーティファクト B（非公開；デプロイ基盤にのみ渡され，Verifier には決して渡されない）.

Certification　このレポートは，「ダイジェスト h_S を持つソースコードが，ダイジェスト $h_{Conf_{\mathrm{Bld}}}$ を持つあるビルド設定の下で，測定値 m_B を持つビルドアーティファクトを生成する」ことの証明書である.

3.2　Attestable Audit（提案コンポーネント）

本アーキテクチャの中核は *Attestable Audit* である．Attestable Auditor は，ソースコードのダイジェストと監査判定（verdict）を結び付ける役割を担う．Attestable Builder と同様に，このコンポーネントもオープンソースかつ再現可能にビルドできるか，又はそのバイナリが公開されていなければならない.

Input　(1) ソースコード S（非公開）；(2) ビルド設定 $Conf_{\mathrm{Bld}}$（非公開）；(3) 監査設定 $Conf_{\mathrm{Aud}}$（公開；例：ファジングバジェット，リンタのルールセット，AI 監査者のプロンプト，形式検証のゴール及び前提など）；(4) 補助入力 Aux（非公開；例：形式的証明）.

Processing　(1) ハッシュダイジェスト h_S，h_{Aux}，$h_{Conf_{\mathrm{Bld}}}$ を計算し；(2) 設定された監査スイート—動的解析（例：ファジング，サニタイザ，Valgrind），静的解析（例：リンティング，テイント解析，Bandit），AI コード監査，形式手法（例：モデル検査，形式証明の検査，強い型付け言語の型検査）—を実行し；(3)（ツール毎の）判定 $Verdict$（公開；例：VERIFIED，TESTED，INCONCLUSIVE，FAILED）を得て；(4) 次のアテステーションレポートを発行する：

$$R_{\mathrm{Aud}} := \mathrm{AR}_{\mathrm{Aud}} \circ H(d_{\mathrm{Aud}}),$$
$$d_{\mathrm{Aud}} := \left(h_S, h_{Conf_{\mathrm{Bld}}}, h_{Aux}, Conf_{\mathrm{Aud}}, Verdict\right).$$

Output　未ハッシュのレポートデータ d_{Aud}（公開）及びアテステーションレポート R_{Aud}（公開）.

Certification　このレポートは，「ダイジェスト h_S を持つソースコードが，ダイジェスト $h_{Conf_{\mathrm{Bld}}}$ を持つビル

© 2026 Information Processing Society of Japan

ド設定のもとで，$Conf_{\mathrm{Aud}}$ で設定された監査を受けたとき，判定 *Verdict* を生成する」ことの証明書である．

3.3 アテステーション手順

Attestable Builder 及び Attestable Auditor は，(1) オープンソースかつ再現可能にビルドできるか，又は (2) それらのバイナリが公開されているとする．これにより，Verifier はそれらのソースコード又はバイナリから測定値 m_{Bld} 及び m_{Aud} を再現できる．

証明書発行．Attester の開発者（Reference Value Provider を兼ねる）は，Attestable Builder 内で Attester のバイナリをビルドし，バイナリ B 及びその証明書 $(d_{\mathrm{Bld}}, R_{\mathrm{Bld}})$ を得る．開発者はまた，Attestable Auditor を用いて Attester のソースコード S を監査し，証明書 $(d_{\mathrm{Aud}}, R_{\mathrm{Aud}})$ を得る．

デプロイ．開発者は B をアテスタブルなプラットフォームにデプロイする．これにより Attester が起動する．

証明書の送付．開発者は，2 つの証明書 $(d_{\mathrm{Bld}}, R_{\mathrm{Bld}})$ 及び $(d_{\mathrm{Aud}}, R_{\mathrm{Aud}})$ を Verifier に送付する．

測定値のアテステーション．Verifier と Attester は通常のアテステーションの手順を実行する．これにより Attester は自身の測定値 m を Verifier に証明する．

ビルドのアテステーション．Verifier は，Attestable Builder の公開リリースから測定値 m_{Bld} を再現し，アテステーションレポート R_{Bld} を TEE の信頼の起点に対して検証し，`REPORT_DATA` フィールドと測定値フィールドがそれぞれ $H(d_{\mathrm{Bld}})$ と m_{Bld} に等しいことを確認する．このステップにより，ダイジェスト h_S を持つソースコードが，ダイジェスト $h_{Conf_{\mathrm{Bld}}}$ を持つあるビルド設定の下で，測定値 m_B を持つビルドアーティファクトを生成することが証明される．

監査のアテステーション．Verifier は，Attestable Auditor の公開リリースから測定値 m_{Aud} を再現し，アテステーションレポート R_{Aud} を TEE の信頼の起点に対して検証し，`REPORT_DATA` フィールドと測定値フィールドがそれぞれ $H(d_{\mathrm{Aud}})$ と m_{Aud} に等しいことを確認する．このステップにより，ダイジェスト h_S を持つソースコード及びダイジェスト $h_{Conf_{\mathrm{Bld}}}$ を持つビルド設定の組が，$Conf_{\mathrm{Aud}}$ で設定された監査を受けたとき，監査判定 *Verdict* を生成することが証明される．

エビデンスの連結．Verifier は，d_{Bld} 及び d_{Aud} が同一の h_S 及び $h_{Conf_{\mathrm{Bld}}}$ を共有すること，並びに d_{Bld} 内の m_B が m に等しいことを確認する．このステップにより，Attestable Builder と Auditor が同一のソースコード S 及びビルド設定 $Conf_{\mathrm{Bld}}$ の組に適用されたこと，並びに Attester が $(S, Conf_{\mathrm{Bld}})$ からビルドされたことが証明される．

3.4 Confidential Computing Proof

Attestable Build と Attestable Audit は，Russinovich *et al.* [21] の提案した *Confidential Computing Proof*（CCP）という一般的概念の下で統一的に捉えることができる．CCP は TEE のアテステーションをハードウェアに裏付けられたゼロ知識証明（ZKP）と見做したものである．すなわち，公開の決定可能（Turing 計算可能）な関係 $\varphi(x, w)$ に対し，オープンソースの TEE アプリケーション Π_φ が公開の入力 x 及び非公開の証拠 w 上にて φ を判定し，レポートデータにステートメント $\exists w.\varphi(x, w)$ をバインドするアテステーションレポートを発行する．このレポート及び測定値の組は，ステートメント $\exists w.\varphi(x, w)$ の ZKP となる．完全な形式化については [21] を参照されたい．

4. 適用例

Section 3 のアーキテクチャは，Attester やアーティファクトの種類に依存しない．Verifier は単にダイジェストを照合するだけであるから，CCP は発行者の TEE 信頼モデルの下で一度発行されれば，全く異なるアテステーション技術を用いる環境にも適用できる．ここでは，Section 1 で概観したアテステーション形態に対応する，3 つの具体例を示す．

4.1 プロプライエタリ SGX アプリケーション

Attester はプロプライエタリな SGX エンクレーブである．CCP はステートレス性，RA-TLS ハンドシェイク [26] を正しく実装していること，機密入力から非信頼出力へのテイントフローの不在といった，エンクレーブの望ましい特性を証明する．SGX エンクレーブイメージのビルドは大規模なツールチェーンを必要とするため，Attestable Builder は CVM 又は CoCo でホストするのが最適である．なお，SGX において，測定値 m_B は `MRENCLAVE` であるため，Attestable Builder は SGX SDK の提供する `sgx_sign` を用いてビルド出力から期待される `MRENCLAVE` を導出する．これが，$\mathrm{meas}(\cdot)$ を $H(\cdot)$ と区別して扱う理由である．Attestable Auditor をデプロイするプラットフォームは，監査の種類・内容に応じて選択する．例えば，AI コード監査は TEE 対応 GPU（例：NVIDIA Confidential Computing）を備えた CVM が推奨されるが，軽量な静的解析は SGX エンクレーブ内でも実行可能である（Section 5）．検証時には，Attester は動作中エンクレーブの `MRENCLAVE` を証明する SGX quote を提示し，ソフトウェアベンダ（従来であれば Reference Value Provider として振る舞う当事者）は，素のリファレンス `MRENCLAVE` の代わりに，事前生成された CCP を提供する．Verifier は Section 3 のプロトコルを実行し，動作中のエンクレーブに対する監査判定が真正なものであることを結論付ける．

© 2026 Information Processing Society of Japan

4.2 Linux サーバ上のプロプライエタリスクリプト

Attester は，TPM を備えた IMA が有効な Linux ホスト上で動作するサーバサイドスクリプトであり，CCP はそのスクリプトの特性を証明する．スクリプトはそのままデプロイされるため，Attestable Builder は省略され，S の測定値は単に IMA が記録するファイルハッシュとなる．Attester は，スクリプトのエントリを含む IMA 測定ログ，当該ログが extend される PCR（通常は PCR 10），及び当該 PCR に対する TPM quote を提示する．ソフトウェアベンダはリファレンスダイジェストの代わりに監査 CCP を提供する．

このケースは Attester のセキュリティモデルと，Attestable Builder/Auditor のセキュリティモデルとの分離を端的に示している．すなわち，監査 CCP は TEE 信頼モデルの下で発行されるが，Attester のデプロイされたプラットフォームは必ずしも TEE である必要はない．Linux OS 自体が CVM 内で動作し，vTPM がゲスト OS とハイパーバイザの双方から隔離されたゲスト特権レベルに配置されている場合 [16] には，デプロイ側の信頼モデルも TEE のそれと一致するが，本アーキテクチャにおいて必須の要件ではない．

4.3 オープンソース推論サーバ上のプロプライエタリ LLM

Attester は CVM 内で動作する推論サーバであり，サーバは完全にオープンソースだが，ロードされるモデルの重みはプロプライエタリである．ここで S は重みファイルであり，CCP はモデルレベルの特性，例えばプロンプトインジェクション耐性ベンチマークにおける定量的スコアを証明する．*Verdict* の形式には一切の制限がないため，そのようなスコアを含めた監査も可能である．ベンチマークには GPU 推論が必要なため，Attestable Auditor は TEE 対応 GPU を備えた CVM で動作する．監査は与えられた重みの特性を証明するものであるため，Attestable Builder は関与しない．検証時には，CVM の起動時測定値及び IMA 測定ログがオープンソースのサーバをカバーし，Verifier はその振る舞いをソースコードレベルで検証できる．測定済みサーバはロードされた重みのダイジェストを計算し，アテステーションエビデンス（例えば `REPORT_DATA`，RTMR，IMA ログ）にバインドする．したがって Verifier は，検証済みのサーバコードを信頼するがゆえに，報告された重みダイジェストも信頼でき，かつソフトウェアベンダの提供する CCP がそのダイジェストをベンチマークの測定結果と紐付ける．

5. Proof-of-Concept Implementation

Attestable Audit の実現可能性を示すため，SGX エンクレーブ内で Python ソースコードを監査し，監査判定をコードダイジェストにバインドする CCP を発行する PoC を実装した．

TEE platform エンクレーブは Gramine [24] 上で動作する．Gramine により，信頼するファイル集合を記述したマニフェストファイル及びアテステーション用エンドポイントの追加のみで，ほぼ無変更の Python スクリプトを SGX 内で実行できる．監査ツールへのアルゴリズム的変更は不要である．デフォルトでは，Microsoft Azure の提供する SGX 対応 VM を SGX ホストとして使用し，アテステーションコラテラルのキャッシュサーバである PCCS として Azure THIM サービスを用いる．

Audit pipeline Auditor エンクレーブは，入力 S として Python スクリプトを受け取り，`ast` モジュールによる構文検査と，Bandit [20] によるセキュリティスキャンを実行する．

CCP binding Section 3 の構成に従い，SGX quote の `REPORT_DATA` は，監査判定 *Verdict* と，与えられた Python スクリプト S の SHA-384 ダイジェストからなる JSON オブジェクト（の SHA-384 ダイジェスト）をバインドする．

Scope 本 PoC は CCP **生成**の手続きをエンドツーエンドで実証する．すなわち，与えられた Python スクリプトの監査判定に `REPORT_DATA` がコミットする，有効な SGX quote を生成する．

6. 関連研究

本節では，提案フレームワークを既存の 3 系統の関連研究と対比し，主たる相違点を述べる．

6.1 Code Transparency Service

Delignat-Lavaud *et al.* [9] は Confidential Computing のための Code Transparency Service（CTS）と呼ばれるアーキテクチャを提案している．CTS は，ソースリリース，ビルド，ポリシー，レビューなどに関する署名付きクレームを公開・追記専用・検証可能な台帳に記録する，透明性あるリリース（transparent release）の基盤である．CTS は，非開示のソースコードに関する監査クレームをどのように生成し，それをアテステーションで観測されるアーティファクトダイジェストにどのように暗号学的に結び付けるかは規定しない．本研究の主たる貢献は，プロプライエタリソフトウェアに対する監査クレーム生成・検証メカニズム（Attestable Audit）を提案したことにある．Attestable Audit により生成された監査クレームの公開・記録・タイムスタンプ付与等に CTS を利用できることから，本研究と CTS とは相補的である．

© 2026 Information Processing Society of Japan

6.2 コード性質のゼロ知識証明

zk-SNARK ／ zk-STARK や RISC Zero [4] のような汎用 zkVM を含む暗号学的ゼロ知識証明（ZKP）は，原理的には CCP を直接置き換えうる．しかし，証明の生成・検証コストと証明サイズは証明対象の検証ロジックの複雑さとともに急激に増大するため [15]，ソフトウェアのビルド，ファジング，GPU アクセラレーションされた AI コード監査のような重いワークロードには現状スケールしない．加えて，検証ロジックの算術制約への書き換え（例：Circom [2]）や信頼されたセットアップ（例：Groth16 [10]）といった実装・信頼モデル上のコストも大きい．

本フレームワークが採用する CCP [21] は，zkVM のように評価トレース全体を暗号学的にエンコードする代わりに，アプリケーションの同一性及び計算結果のみをハードウェア RoT に紐付く鍵で署名する．証明の健全性がハードウェア RoT，TEE の隔離，及び測定されたアプリケーションロジックの正しさに依存するという信頼モデル上の代償を払う．その対価として既存のビルドシステムや監査パイプラインを TEE オーバーヘッドの範囲内でほぼ無変更に実行できる．それゆえ，追加の計算コストは，メモリ暗号化などの隔離保護機能による定数倍のコストと，アテステーションレポート発行のコストのみである．

6.3 Property-Based Attestation 及び Proof-Carrying Code

バイナリの同一性ではなくコードの**性質（*property*）**をアテステーションするという目標は，Semantic Remote Attestation [11]，Property-Based Attestation（PBA）[5], [6], [22]，及び Proof-Carrying Code（PCC）[1], [17], [18] と共有するものである．主たる相違点はコードの非開示性にある．

PBA では，許容可能な各構成が当該特性を持つことを立証するために，認証者による検査 [5] 又は合意されたリストの公開 [6] を通じて，誰かがコードを検査できることが前提である．また，PCC では，特性の機械検証可能な証明がバイナリとともに配布されるため，事実上コードが開示される．

一方，Attestable Audit では，Verifier には，ダイジェスト，監査設定，監査判定のみが渡り，ソースコードやビルドアーティファクトなどのプロプライエタリなデータは渡らない．構造的には，Attestable Audit は PBA における Trusted Third Party（TTP）を透明で機械検証可能な TEE に置き換えたものと見做せる．CCP は測定された監査パイプラインが実際に監査対象コードに対して実行されたことまで証明する点で，認証者による署名の事実のみを証明する TTP のクレデンシャル [5] よりも強い保証を与える．また，ソースレベルの特性を実行時のバイナリにバインドする際，PBA が実質的に要求する再現可能ビルド [14] 及びソース開示は，(h_S, m_B) の対応をハードウェア RoT に紐付く署名の下で証明する Attestable Build により不要となる．

なお，これらのアプローチは相補的でもある．PCC スタイルの証明を非公開の補助入力 *Aux* として Attestable Auditor 内で検査すれば，コードも証明も開示することなく，証明水準の保証を Verifier に伝えられる．これは Attestable Auditor として形式検証を利用する場合と全く同じものである．

7. 結論

Attestable Build 及び Attestable Audit という 2 つの TEE アプリケーションを組み合わせることにより，プロプライエタリなソフトウェアに対するアテステーションにおける根本的なギャップ—すなわち，測定値をリファレンス値と照合しても，Verifier に分かるのは「このダイジェストを持つ何かが動作している」ことだけであり，Reference Value Provider への全面的な信頼を強いられるという問題—を解消した．これらの TEE アプリケーションは，ソースコードもビルドアーティファクトも Verifier に開示することなく，ソースコードダイジェストをビルドアーティファクトの測定値及び監査判定とバインドする．これにより，コードを開示することなく，プロプライエタリなアプリケーションに対するプロパティベースアテステーションが可能となる．Gramine を用いた Intel SGX 上で Bandit により Python コードを監査する PoC 実装を与え，本アーキテクチャの実現可能性を実証した．

参考文献

[1] A. W. Appel, “Foundational proof-carrying code,” in *Proceedings of the 16th Annual IEEE Symposium on Logic in Computer Science*, ser. LICS ’01. USA: IEEE Computer Society, 2001, p. 247.

[2] M. Bellés-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. Baylina, “Circom: A circuit description language for building zero-knowledge applications,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 6, pp. 4733–4751, 2023. [Online]. Available: https://doi.org/10.1109/TDSC.2022.3232813

[3] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, “Remote attestation procedures (RATS) architecture,” RFC 9334, IETF, Jan. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9334

[4] J. Bruestle, P. Gafni, and the RISC Zero Team, “RISC Zero zkVM: Scalable, transparent arguments of RISC-V integrity,” 2023. [Online]. Available: https://dev.risczero.com/proof-system-in-detail.pdf

[5] L. Chen, R. Landfermann, H. Löhr, M. Rohe, A.-R. Sadeghi, and C. Stüble, “A protocol for property-based attestation,” in *Proceedings of the First ACM Workshop on Scalable Trusted Computing*, ser. STC ’06. New York, NY, USA: Association for Computing Machinery, 2006, pp. 7–16. [Online]. Available: https://doi.org/10.1145/1179474.1179479

[6] L. ”Chen, H. Löhr, M. Manulis, and A.-R. Sadeghi, “Property-based attestation without a trusted third

© 2026 Information Processing Society of Japan

party," in *Information Security*, T.-C. Wu, C.-L. Lei, V. Rijmen, and D.-T. Lee, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 31–46. [Online]. Available: https://doi.org/10.1007/978-3-540-85886-7_3

[7] Cloud Native Computing Foundation, "Confidential containers," gitHub repository. [Online]. Available: https://github.com/confidential-containers/confidential-containers

[8] V. Costan and S. Devadas, "Intel SGX explained," Cryptology ePrint Archive, Report 2016/086, 2016. [Online]. Available: https://eprint.iacr.org/2016/086

[9] A. Delignat-Lavaud, C. Fournet, K. Vaswani, S. Clebsch, M. Riechert, M. Costa, and M. Russinovich, "Why should I trust your code?" *Commun. ACM*, vol. 67, no. 1, pp. 68–76, Dec. 2023. [Online]. Available: https://doi.org/10.1145/3624578

[10] J. Groth, "On the size of pairing-based non-interactive arguments," in *Advances in Cryptology – EUROCRYPT 2016*, M. Fischlin and J.-S. Coron, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2016, pp. 305–326.

[11] V. Haldar, D. Chandra, and M. Franz, "Semantic remote attestation—a virtual machine directed approach to trusted computing," in *3rd Virtual Machine Research & Technology Symposium (VM 04)*. San Jose, CA: USENIX Association, May 2004. [Online]. Available: https://www.usenix.org/conference/vm-04/semantic-remote-attestation%E2%80%94-virtual-machine-directed-approach-trusted-computing

[12] D. Hugenroth, M. Lins, R. Mayrhofer, and A. R. Beresford, "Attestable Builds: Compiling verifiable binaries on untrusted systems using trusted execution environments," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 4514–4528. [Online]. Available: https://doi.org/10.1145/3719027.3765128

[13] Intel, "Intel TDX virtual firmware design guide," 2023, last retrieved: 2026-05-25. [Online]. Available: https://www.intel.com/content/www/us/en/content-details/733585/intel-tdx-virtual-firmware-design-guide.html

[14] C. Lamb and S. Zacchiroli, "Reproducible Builds: Increasing the integrity of software supply chains," *IEEE Software*, vol. 39, no. 2, pp. 62–70, 2022. [Online]. Available: https://doi.org/10.1109/MS.2021.3073045

[15] A. Nainwal, A. Kamble, and N. Awathare, "A comparative analysis of zk-SNARKs and zk-STARKs: Theory and practice," 2025, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2512.10020

[16] V. Narayanan, C. Carvalho, A. Ruocco, G. Almasi, J. Bottomley, M. Ye, T. Feldman-Fitzthum, D. Buono, H. Franke, and A. Burtsev, "Remote attestation of confidential VMs using ephemeral vTPMs," in *Proceedings of the 39th Annual Computer Security Applications Conference*, ser. ACSAC '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 732–743. [Online]. Available: https://doi.org/10.1145/3627106.3627112

[17] G. C. Necula, "Proof-carrying code," in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL '97. New York, NY, USA: Association for Computing Machinery, 1997, pp. 106–119. [Online]. Available: https://doi.org/10.1145/263699.263712

[18] ——, "Compiling with proofs," Ph.D. dissertation, Carnegie Mellon University, 1998.

[19] N. Nethercote and J. Seward, "Valgrind: a framework for heavyweight dynamic binary instrumentation," *SIGPLAN Not.*, vol. 42, no. 6, pp. 89–100, Jun. 2007. [Online]. Available: https://doi.org/10.1145/1273442.1250746

[20] PyCQA, "Bandit," gitHub repository. [Online]. Available: https://github.com/PyCQA/bandit

[21] M. E. Russinovich, C. Fournet, G. Zaverucha, J. Benaloh, B. Murdoch, and M. Costa, "Confidential Computing Proofs: An alternative to cryptographic zero-knowledge," *ACM Queue*, vol. 22, no. 4, Sep. 2024. [Online]. Available: https://queue.acm.org/detail.cfm?id=3689949

[22] A.-R. Sadeghi and C. Stüble, "Property-based attestation for computing platforms: caring about properties, not mechanisms," in *Proceedings of the 2004 Workshop on New Security Paradigms*, ser. NSPW '04. New York, NY, USA: Association for Computing Machinery, 2004, pp. 67–77. [Online]. Available: https://doi.org/10.1145/1065907.1066038

[23] R. Sailer, X. Zhang, T. Jaeger, and L. van Doorn, "Design and implementation of a TCG-based integrity measurement architecture," in *Proceedings of the 13th USENIX Security Symposium, August 9-13, 2004, San Diego, CA, USA*, M. Blaze, Ed. USENIX, 2004, pp. 223–238. [Online]. Available: http://www.usenix.org/publications/library/proceedings/sec04/tech/sailer.html

[24] The Gramine Project, "Gramine library OS with Intel SGX support," gitHub repository. [Online]. Available: https://github.com/gramineproject/gramine

[25] C.-c. Tsai, D. E. Porter, and M. Vij, "Graphene-SGX: A practical library OS for unmodified applications on SGX," in *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. Santa Clara, CA: USENIX Association, Jul. 2017, pp. 645–658. [Online]. Available: https://www.usenix.org/conference/atc17/technical-sessions/presentation/tsai

[26] R. Walther, C. Weinhold, and M. Roitzsch, "RATLS: Integrating transport layer security with remote attestation," in *Applied Cryptography and Network Security Workshops: ACNS 2022 Satellite Workshops, AIBlock, AIHWS, AIoTS, CIMSS, Cloud S&P, SCI, SecMT, SiMLA, Rome, Italy, June 20–23, 2022, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2022, pp. 361–379. [Online]. Available: https://doi.org/10.1007/978-3-031-16815-4_20

付　　録

A.1 Code availability

Attestable Auditor の PoC 実装は `https://github.com/acompany-develop/attestable-audit-poc` より利用可能である.

© 2026 Information Processing Society of Japan